

Transactions let the application pretend there are no crashes (atomicity), that nobody else is concurrently accessing the DB (isolation), and that storage devices are perfectly reliable (durability) → these things happen but transactions are an abstraction that hides those problems so the app doesn't have to worry about it



In this same way, Consensus is an abstraction that allows applications to ignore some problems with distributed systems

Consistency Guarantees:

Eventual Consistency is a very weak guarantee, and is also hard to work with; Bugs are often subtle and hard to find by testing because the edge cases of eventual consistency only become apparent when there is a fault in the system.

Linearizability:

What if DB could give illusion that there is only one replica even if there are multiple?

This is the idea behind **linearizability** (**atomic consistency**)



recency guarantee: as soon as one client makes a write, all clients reading from the DB must be able to see the value written

What makes a system linearizable?

Basic idea: make it appear as if there is only a single copy of the data

In a linearizable system, we imagine that there must be some point in time (between the start and end of the write operation) at which the value of the **register** (one key in a KV store, one row in a relational database, or one document in a document database) atomically flips to the new value.

∴ If one client's read returns the new value, all subsequent reads must also return the new value, even if the write operation is incomplete.

Relying on Linearizability:

When Linearizability is useful?

Locking and leader election:

- systems that use single leader replication need to avoid split-brain

One way of doing this is to use a lock; the node with the lock is the leader node. No matter how this lock is implemented, it must be linearizable: all nodes must agree which node owns the lock, otherwise it is useless.

Apache ZooKeeper and etcd are coordination systems used to implement distributed locks and leader election — they use consensus algos to implement linearizable operations in a fault tolerant way.

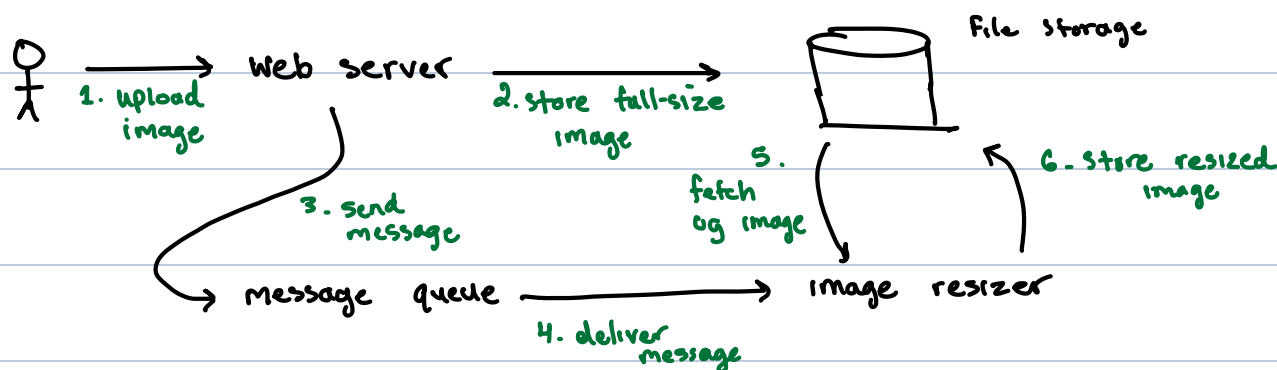
Constraints and Uniqueness Guarantees

- common in DB's, ex. a username or email address must uniquely identify one user

If you want to enforce this constraint as the data is being written, you need linearizability.

A hard uniqueness constraint requires linearizability.

Cross-channel Timing Dependencies



If file storage is not linearizable here, risk of race condition where image resizer see's old picture or nothing at all from file storage

Implementing Linearizable Systems :

- single-leader replication has the potential to be linearizable
- consensus algorithms are linearizable (Zookeeper, etcd)
- leaderless replication does not provide linearizability
- multi-leader replication does not provide linearizability

The CAP Theorem

Linearizability Trade-off :

- if your application requires linearizability, and some replicas are disconnected due to network failures, then those nodes must become unavailable
- if your application does not require linearizability, then those nodes can remain available, but its behavior is not linearizable.

↓
more tolerant of
network problems

This insight is popularly known as the **CAP Theorem**

↙
either Consistent or Available
when Partitioned

Linearizability is really slow, a faster algorithm doesn't exist, but weaker consistency guarantees can be much faster; this trade-off is important for latency sensitive systems

Ordering and Causality

Causality imposes an ordering on events: cause comes before effect

If a system obeys the ordering imposed by causality, you call it

causally consistent

With Linearizability, we have a **total order** of operations; for any two operations we can always say which happened first

With Causality, we have a **partial order** of operations; two events are ordered if they are causally related, but are incomparable if they

are concurrent.

Causal consistency is the strongest possible consistency model that does not slow down due to network delays and remains available in the face of network failures.

Sequence Number Ordering

- Keeping track of all causal dependencies can become impossible.

(large overhead)

Instead, we use a logical clock to generate monotonically increasing timestamps that are assigned to every operation



these provide causal order as if operation A depended on operation B, we know $\text{number}(A) > \text{number}(B)$; concurrent operations will be arbitrarily labeled

In single-leader replication, the replication log defines the total order of write operations that is consistent with causality → increment a counter for each operation and assign its value to each operation

Lamport Timestamps:

Each node has unique id and keeps a counter of the number of operations it has processed, the timestamp is then just:

(counter, unique id)

the counter is shared between all nodes, so if two nodes process an operation concurrently it's possible they have the same counter value.

If node 1 processes 5 operations (counter = 5), then node 2 processes something, its counter will be 6.

↓
counter value is included in write request

Timestamp ordering is not sufficient:

Ex. In the situation where a username needs to uniquely identify a user, if two users concurrently try to create an account with the same username, one should succeed and one should fail.

At first glance, the approach of picking the one with the lower timestamp as the winner should be sufficient. The problem is this only works after the fact: once you have collected all the username creation operations in the system. It doesn't work when the node just received a request to use that username and needs to decide now if it should succeed or fail because it does not know if another node is in the process of creating account with the same username.

In order to implement a uniqueness constraint, you need total ordering and you need to know when the order is finalized. Node must be sure that no other node can insert a claim for the same username ahead of its operation in the total order

total order broadcast - idea of knowing when the total order is finalized

Total Order Broadcast

Single-leader replication determines the order of operations by choosing one leader and sequencing all operations on a single CPU.

The challenge of scaling the system if the throughput is greater than what a single leader can handle is known as **total order**

broadcast or **atomic broadcast**



is usually described as a protocol for exchanging messages between nodes, and it requires two safety properties are always satisfied:

1. **Reliable delivery** - no messages are lost; if a message is delivered to one node, then its delivered to all nodes

2. **Totally ordered delivery** - messages are delivered to every node in the same order

Consensus services such as Zookeeper and etcd implement total order broadcast

total order broadcast is what you need for database replication:

if every message represents a write, and every replica processes these writes in the same order, then replicas will remain consistent

→ known as state machine replication

Distributed Transactions and Consensus

Consensus is one of the most important and fundamental problems

of distributed systems

goal is simply to get several nodes to agree on something

Situations where it is important for nodes to agree:

- leader election

- atomic commit - in dB that support transactions spanning several

nodes/partitions, the transaction may fail on some

node/partitions and succeed on others. To maintain

atomicity, must get nodes to agree on the outcome

of it; either they all commit (no fails) or all

rollback/abort (some fails)

Atomic and Two-Phase Commit

The outcome of a transaction is either a commit or an abort;

Transaction atomicity prevents half-finished transactions from littering the dB with half finished results and half-updated state.

Introduction to two-phase commit

2PC is an algorithm for achieving atomic transaction commit across multiple nodes

2PC uses a **coordinator** (aka **transaction manager**)
↑ either a dedicated service or one of the existing services acts as the coordinator

When 2PC is used, a distributed transaction begins with the application reading and writing data on multiple database nodes, as normal

↓
call these nodes
Participants

When ready to commit, the coordinator begins phase 1:

Coordinator sends a prepare request to all participants, asking if they are able to commit; then tracks the response from all participants

- if all participants respond with "yes", then the coordinator sends out a "commit" request in phase 2, and the commit takes place

- if any of the participants reply "no", the coordinator sends an abort request to all nodes

Surely the prepare and commit requests can just as easily be lost in 2PC like in 1PC across several nodes. What makes 2PC different?

To understand, we must break down the 2PC process in more detail:

1. When application wants to begin a distributed transaction, it must request a globally unique transaction ID from the coordinator.

2. The application begins single-node transaction on each of the participants, and attaches this transaction ID to them.

If anything goes wrong at this stage, all the participants can abort.

3. When the application is ready to commit, it sends the "prepare" request to all participants, tagged with the transaction ID. If any of the requests fail or time out, the coordinator sends "abort" request tagged with transaction ID to all participants.

4. When participant gets "prepare" request, it ensures that it

crash/power failure is not acceptable excuse for refusing to write all transaction data to disk. By replying "yes"

commit later.

to coordinator, it promises to commit the transaction without error if requested,

5. When coordinator receives responses to all prepare requests, it must make a decision on whether to commit and write that decision to the transaction log on disk so it knows how it decided incase it crashes. This is called the **commit point**.

6. Once the coordinator has written its decision to disk, the "commit" or "abort" request is sent to all participants. If this request fails or times out, it must retry forever until it succeeds.

∴ the protocol contains two points of no return:

1. If the participant voted yes, it promises it will definitely be able to commit later

2. Once the coordinator decides, that decision is irrevocable

Coordinator failure

If a coordinator fails before sending a prepare request, the participants can safely abort.

Once a participant has replied "yes" to a prepare request,

it can no longer abort unilaterally, it must wait to hear back from the coordinator. If the coordinator crashes at this point, the node must wait. A transaction in this state is called **in doubt** or **uncertain**.

The only way for 2PC to complete is to wait for the coordinator to recover, and determine status of all uncertain transactions by reading transaction log. Any uncertain transactions that do not have a commit record in the log are aborted.

Distributed Transactions in Practice

Distributed transactions have a mixed reputation:

- provide important safety guarantee
- cause operational problems
- kill performance

Two different types of distributed transactions:

1. **Database-internal distributed transactions** - distributed DBs that (use replication and partitions as a standard) support internal transactions among the nodes; all nodes running the same dB software

2. **Heterogeneous Distributed Transactions** - participants are from two or

more different technologies. Ex. two dB from different vendors
alot more or even non-dB systems like message brokers
challenging

Exactly once message processing

Ex. A message from a message queue can be acknowledged as processed iff the dB transaction for processing the message was successfully committed → atomically committing the message acknowledgement and dB writes in a single transaction

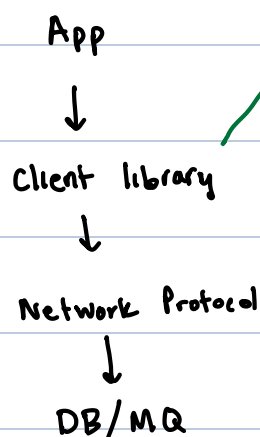
XA transactions

XA is a standard for implementing 2PC across heterogeneous technologies.

- It is NOT a network protocol, rather it is a "C API" for interfacing with a transaction coordinator
- Bindings for this API exist in other languages, ex. Java.

XA assumes that your application uses a network driver or client library to communicate with participants.

TM and client library usually live on the application process itself



implements XA protocol on behalf of the application

Your app does not speak XA directly, instead:

- Transaction Manager (TM) speaks XA to the Resource Managers
- Client library exposes XA hooks so TM can control the resource

362